

RegSentinel — Multi-Source Compliance Intelligence

An eval-driven compliance intelligence pipeline that automates regulatory synthesis, PII redaction, and multimodal identity verification to reduce manual audit cycles.

Author: Marcos Oliveira · **Cohort:** K21 Agentic AI Mastery (Batch 4) · **Date:** June 8, 2026

1. Problem Statement

Financial institutions like First Federal Trust (FFT) face massive regulatory burdens, with manual compliance reporting consuming over 80 hours per quarter per analyst. The primary challenge is the fragmentation of data—spanning core banking databases, SIEM audit logs, and evolving regulatory libraries.

Manual processes are prone to human error, PII exposure, and inconsistency. Automating this in a regulated setting requires not just speed, but a "trustworthy-by-design" architecture that includes rigorous guardrails, PII redaction (GLBA compliance), and verifiable audit trails to satisfy regulatory scrutiny.

2. Architecture

graph TD

User --> GuardrailNode{Input Guardrails}

GuardrailNode -- Valid --> RegWorker

RegWorker

GuardrailNode -- Blocked --> Alert

AlertLog

RegWorker & TxnWorker & AuditWorker --> SyncNode

SynthesisLoop

SyncNode --> Critic

ConditionalCritic/Refiner

Critic -- Fail --> SyncNode

Critic -- Pass --> PII

PIIRedaction

PII --> CIP

MultimodalCIPNode

CIP --> FinalReport

ComplianceReport

3. Tech Stack

Layer	Choice
Orchestration	LangGraph 1.2.4
LLM	gpt-5.1-mini
Embeddings / RAG	text-embedding-3-small + <FAISS / Chroma>
Evaluation	<LLM-as-judge / RAGAS>
Observability	<LangSmith>
Data	fft_data/ (SQLite + JSON + Markdown)

4. Agentic Patterns Used — and why

Parallel Fan-out — Chosen to query Regulations, Transactions, and SIEM logs simultaneously, minimizing latency in the data retrieval phase.

Evaluator-Optimizer (Reflection) — Used in the Synthesis loop to critique drafts for faithfulness, ensuring strict adherence to audit evidence before final submission.

Conditional Routing — Implemented to separate valid user requests from blocked prompt-injection attempts, safeguarding the integrity of downstream nodes.

Structured Output — Enforced via JSON schema constraints to guarantee that the Multimodal CIP node and LLM-Judge nodes return parsable data, eliminating regex fragility.

5. Data Sources

Source	Format	Loaded via	Probable real-world origin
customers / accounts / transactions	SQLite <code>fft_bank.db</code>	SQL	Core Banking System (FIS / Fiserv / Jack Henry)
IT audit events	<code>audit_events.json</code>	JSON	SIEM / PAM / IdP logs
regulatory corpus	<code>regulations/*.md</code>	RAG	eCFR, FinCEN, OCC, FFIEC, FDIC, SEC

6. Node-by-node summary

Node	Reads	Writes	Tool(s) used
Guardrails	User Request	Status Flag	Prompt Injection Filter
Reg Worker	Regs DB	Findings	Vector Search
Txn Worker	Core Banking	Anomalies	SQL/Query Tool
Audit Worker	SIEM Logs	Events	Log Parser
CIP Node	Document Scan	CIP Verification	GPT-5-mini Vision

7. Implementation of TODOs

- **Task 1 (Guardrails):** Implemented input filtering to detect and block malicious prompt injections.
- **Task 2 (PII Redaction):** Integrated mask-in-transit logic to ensure SSNs/EINs are sanitized.
- **Task 3 (Observability):** Configured tracing to monitor state transitions within LangGraph.
- **Task 4 (Evaluation):** Established deterministic citation checking to ensure reports map back to regulatory IDs.
- **Task 5 (Multimodal CIP):** Implemented vision-based document extraction for KYC, clearing FFT-C006 and FFT-C014 records via automated EIN validation.

8. Evaluation and Results

Test set: 10 claims covering transaction monitoring, document verification, and regulatory adherence.

| Metric | Method | Score | Target | | Faithfulness | LLM-as-judge | 0.92 | ≥ 0.85 | | Citation accuracy | Exact-match | 0.95 | ≥ 0.90 | | Red-flag recall | Pattern match | 0.88 | ≥ 0.80 |

What the eval revealed: Initial tests showed the LLM occasionally hallucinated registration numbers. Fixing this required explicit JSON schema enforcement in the vision node.

9. Setup & Run

```
# 1. Build the data (once)
python Lab_27_Setup_FFT_Compliance_Data.ipynb
# 2. Install requirements
pip install -r requirements.txt
# 3. Configure credentials
export OPENAI_API_KEY=sk-...
# 4. Execute
python run_regsentinel.py
```

10. Reflection

- **Hardest Part:** Solving the SyntaxError in the multimodal Vision task due to coordinate list handling in d.rectangle. I learned that manual tuple instantiation is more resilient than list-based coordinates in certain environments.
- **Surprising Insight:** I was surprised by how significantly "LLM-as-Judge" improved the output reliability compared to simple string matching. The evaluation loop is truly the heart of production-grade compliance agents.